



Course Project: CSAI 203, Introduction to Software Engineering

Project Objective

The purpose of this course project is to provide students with hands-on experience in applying fundamental software engineering practices. By working in teams, students will learn how to define, design, implement, and deliver a software product that meets real-world requirements. The project also aims to improve collaboration, communication, and documentation skills, preparing students for future software development work.

By completing this project, students will be able to:

- Apply the software development life cycle (SDLC) phases in practice.
- Formulate a problem statement and define the requirements of the software.
- Prepare proposal and design documents using appropriate notations.
- Practice team collaboration and version control (e.g., Git, GitHub).
- Implement a software system incrementally, following good coding practices.
- Deliver a functional software application with partial features (mid-project).
- Practice project management skills, including phase planning and progress tracking.
- Prepare technical and user documentation to support the deployment and use of the software.
- Deploy and demonstrate a working software solution.

Project Phases Timeline

Project Phases	Deliverables	20 Grades	Deadline
Project Announcement & Group Formation	Team list, Chosen project topic/title and git repo URL	One grade penalty if not submitted on time	Saturday 11/Oct/2026: 11:59 PM
Requirements Specification Document	SRS documents describing what the software will do before moving to design and implementation.	20% (4 Grades)	TBD
Design Document	Design document including the different diagrams defining how the system will be built, etc.	25% (5 Grades)	TBD
Core Functionality Prototype	Core features working - about 50% of its intended functionality..etc.	25% (5 Grades)	TBD
Final Delivery & Presentation	Fully functional application with 100% of features implemented, deployment, presentation, etc.	30% (6 Grades)	TBD

Project Phases Details:

Phase 1: Project Announcement & Group Formation

- **Deliverable:**
 - Team list
 - Chosen project topic/title
 - Each group must select a project idea
 - A form will be created for students to submit their group members and ideas.
 - Members are highly encouraged to be from the same lab group.
 - Create a GitHub repository and share access to it with your assigned TA.
-

Phase 2: Software Requirements Specification (SRS)

[The details, and requirements of this Phase will be added later as more course materials are covered.]

Phase 3: Design Document

[The details, and requirements of this Phase will be added later as more course materials are covered.]

Phase 4: Core Functionality Prototype

[The details, and requirements of this Phase will be added later as more course materials are covered.]

Phase 5: Final Delivery & Presentation

[The details, and requirements of this Phase will be added later as more course materials are covered.]

Bonus [Optional]

[The details, and requirements of this Bonus section are optional. Dr.Rakha may add details later as more course materials are covered.]

Team Formation Phase:

- The number of students per team is **3 - 4 students**.
- Oversized groups are not allowed.
- Team members are encouraged to be from the same lab. Team members can be from different LABS. But NOTE that: It will be your FULL RESPONSIBILITY to attend the discussion. Any student who misses the scheduled discussion will automatically receive ZERO.

- Each group will assign a representative student as the primary point of contact for communication with TAs.
- The deliverables discussion will take place in the lab after the online submission deadlines.

Note: Workload distribution should be vertical. A student is expected to work across all layers in the software engineering life cycle.

Project Idea Selection:

Student groups must select one of the suggested ideas below:

- **Academic Course & Schedule Planner:** A web app where students build semester conflict-free timetables, track prerequisite requirements, and calculate GPAs, while admins manage course offerings, capacities, and prerequisites.
- **Campus Event & Student Club Hub:** A platform for student organizations to publish campus events, manage member registrations, issue digital tickets, track event attendance, and manage club budgets.
- **Internship & Career Portal:** A mini-job board connecting students with recruiters, allowing students to submit applications and track status updates while employers post listings and manage interview schedules.
- **Smart Health & Fitness Tracker:** A wellness application where users log daily workouts, track meal calories against customizable goals, view progress charts, and share workout routines.
- **Student Peer Exchange & Rental Marketplace:** A campus marketplace where students buy, sell, or rent used textbooks and lab hardware, featuring item listings, availability calendars, booking requests, and user ratings.
- **AI Prompt Library & Workbench:** A community platform tailored for AI students to store, search, rate, and share effective prompt templates, featuring tag categorization, favorite collections, and a sandbox viewer to test variable inputs.
- **Doctor Appointment & Patient Portal:** A healthcare booking platform where patients search for medical specialists, reserve consultation slots, and view medical history, while doctors manage daily schedules and issue prescriptions.
- **Online Quiz & Auto-Grading System:** An educational platform where instructors build question banks, create timed quizzes, and view grade distributions, while students take assessments and receive instant auto-graded feedback.
- **Shared Expense & Budget Tracker:** A financial management tool that lets users track personal daily spending, split shared bills among roommates or team members, set monthly category limits, and view spending analytics.
- **Team Task & Kanban Management Board:** A collaboration tool where project teams organize tasks into customizable Kanban stages (To Do, In Progress, Done), assign team members, set due dates, and track project activity logs.

Expected Project Scope:

- Functional Requirement Features (20 required):

- [The details, and requirements of this Phase will be added later as more course materials are covered.]

- Non-Functional Requirement Features (5 required):

- [The details, and requirements of this Phase will be added later as more course materials are covered.]

GitHub Repository Structure

- Each team must maintain a single GitHub repository for their project.
- The repository must follow a clear structure, with at least the following folders:
 - **/docs** → Contains all submitted documents (SRS, Design, Reports, Testing Evidence, etc.).
 - **/src** → Contains all source code.
 - **/tests** → Contains test cases (unit tests, manual test reports if applicable).
 - **/deployment** → Contains deployment instructions, configuration files, and environment setup scripts (if any).
- A **README.md** file must be present in the root of the repository, clearly describing:
 - Project title and team members
 - Short project description
 - Instructions for setting up and running the application
 - Links to relevant documents in /docs
- Commits must be meaningful (e.g., “Added login feature” instead of “fix”) and reflect contributions from all members.

Important Rules:

1. The number of students per team is **3 to 4 students**.
2. Teams are encouraged to be from the same lab. Team members can be from different LABS, but NOTE that: It will be your FULL RESPONSIBILITY to attend the evaluation session. Any student who misses the scheduled evaluation session will automatically receive ZERO.

3. Projects are not allowed to be a reuse of another course project.
4. If a student is retaking this course, they must select a new project idea different from their previous one.
5. For each phase, **if a late submission is permitted, a 25% penalty will be incurred** for each day after the deadline. After two overdue days, no submissions will be accepted.
6. Each phase's deliverables must be uploaded to the Moodle assigned assignment and pushed to the repository **before the submission deadline**.
7. The deliverable version uploaded to the Moodle assignment is the one that will be evaluated. The GitHub version will just be a reference to track progress.
8. **Submissions that do not follow the cover page and naming convention may receive up to a 10% penalty for that phase.**
9. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. **GitHub is mandatory in the evaluation process starting Phase-2** and must follow the "GitHub Repository Structure" mentioned. Failing to maintain an updated GitHub repository for your projects may result in a penalty of **up to 20%** for the phase under evaluation.
10. All team members are expected to contribute fairly across the project. Unequal contribution may result in individual grade deductions.
11. **Although this is a team project, every member is expected to fully understand all submitted work. TAs may ask any student about any part of the deliverable (diagrams, code, documentation, etc). Any student unable to explain their project/code may lose marks individually, even if the deliverable was submitted.**
12. **Cheating** could result in a negative grade (up to - 10 Course Grades).
13. If your project idea includes Artificial Intelligence (AI), you are NOT required to train models from scratch. You are only allowed to use pre-trained models to implement the intended feature(s). Efforts spent on training models will not be considered in the evaluation.
14. **Academic Integrity Reminder:** Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct (Up to cheating). Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. You may use AI tools to support your learning or improve your understanding, but the final content must be genuinely your own. All submissions/deliverables are subject to plagiarism and AI-detection checks. If there is reasonable suspicion of AI-generated or copied content, the lecturer/TA reserves the right to:
 - Assign a penalty for **cheating** if misuse is confirmed.
 - Request additional work or conduct an oral examination to verify authorship and understanding of the submitted material.
15. Each phase submission (when needed) must include **clear documentation** as required (requirements, design diagrams, progress reports, testing evidence, etc.).
16. A **final presentation and live demo** are mandatory unless canceled by the course instructor (Failing to do so may lead to a big % penalty).
17. The project must be implemented using only **requested frameworks/technologies (Backend) + (Frontend)**.
18. The requirements for the frontend UI are **standard** using UI design tools such as Figma is considered for bonus grades. Please ensure that your non-functional requirements fall within these limitations.
19. Auto-code generation tools (such as UI Design to Code or System Design to Code, or Diagrams to Code) are not permitted for use and will be considered **cheating**.
20. Grading focuses on **software engineering practices** (requirements, design, implementation, design patterns, testing, documentation), not just mastering a specific coding technology.

21. The Model–View–Controller (MVC) pattern is **mandatory** and must be clearly reflected in both the Design Document and the code structure of the project. Teams are required to describe how MVC is applied (Model, View, and Controller responsibilities) as part of their Design Document.

Grading Policy

- Understanding deliverables and their details will be evaluated individually for each team member.
- Your project GitHub is part of the evaluation weight.
- All the mentioned requirements in each phase are part of the evaluation criteria weights.

Deliverable Discussion/Response Policy:

- If a student is asked a question during evaluation and does not answer, 5 points will be deducted.
- If the student fails to answer a second question, an additional 5 points will be deducted.
- If the student fails to answer a third question, the student will receive 0 for the Individual Understanding mark.
- If the student fails to answer a fourth question, the student will receive 0 for the deliverable itself, regardless of the team's mark

Delivery Guide:

- The GitHub repository for the project will be used to evaluate the team's collaboration and each team member's effort. The TA will evaluate the scale of commits and contributions to evaluate each team member's contributions. Make sure to distribute development work evenly across your team. It is essential to distribute the software requirements across the team members to ensure an even workload for this Phase. Make sure to share your project GitHub repository with your assigned TA.
- Note that GitHub activity **during the work period and contribution tracking will be essential parts** of the evaluation for the implementation phases. Each team member must work on their own branch, and all branches should be merged appropriately (with Pull Requests and good descriptions) before submitting each phase's deliverable. Fair and consistent contribution from every team member is expected in the evaluation. GitHub will track each team member's contributions. **Use descriptive commit and PR messages.**
- **Note: In the delivery discussion of the bonus, each group must show updated Class and Sequence Diagrams, including the impact of the design patterns above.**
- Commits tracking will be required in your evaluation. Don't delete any of your work branches; if you do, you will lose a big grade percentage.
- **As with previous deliverables,** you must also upload the deliverable package, which in this case includes the source code/+ any data, schema, and configuration files. It is expected that the uploaded content matches what is on your team's GitHub repository.
- The Model–View–Controller (MVC) pattern is **mandatory** and must be clearly reflected

Document Submission, Cover Page & Naming Convention

- All submitted documents (SRS, Design, Reports, etc.) must have a cover page and follow a strict naming convention:
CourseCode_ProjectPhase_TeamID_RepresentativeID.pdf
- **Example:**
CSAI203_SRS_Team05_20231234.pdf
CSAI203_Design_Team05_20231234.pdf
- TeamID will be assigned at the time of project announcement.
- RepresentativeID is the University ID of the team representative.

Project Deliverable Documents

All project deliverables should have the following cover page:

Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2026

Introduction to Software Engineering
<Project Name>

<Report Title (Deliverable Description)>
Team Number: #
Team Members:
<Name> <ID>

Representative Contact info:

Write one email to be able to contact your team.

<Date>

○